Grammar Matters More Than Graphs: Training-Free Bug Localization with Sub-4B Language Models on a 4 GB Consumer GPU

Yukteswar

https://github.com/yukteswar23/slm-bug-localizer

November 2026

Abstract

We ask how far scaffolding alone can push off-the-shelf language models of at most 4B parameters at *bug localization*, finding the files and functions that a GitHub issue requires to change, when the models run fully offline on a laptop with a 4 GB GPU and receive no training. We build one ReAct-style agent harness with file tools, code-graph tools, an Evidence Buffer, behavioural guards and grammar-constrained decoding, and ablate it on SWE-bench Lite with two model families, Gemma 4 E2B and Qwen3.5-4B (4-bit, llama.cpp). Over 1,000 test episodes (100 tasks $\times$ 5 configurations $\times$ 2 models, 13.3 GPU-hours) and a 23-task development split, we find: (i) grammar-constrained decoding is the single most important component; removing it costs Qwen3.5-4B 46 points of File@1 (85% $\rightarrow$ 39%; 46 tasks lost, none gained, exact McNemar $p < 10^{-13}$), mostly because the model drops the tool name from its final answer; (ii) the *form* of the answer matters as much as the search: a structured answer format, a repeated-call guard, budget hints and ranked lists raise Qwen3.5-4B's Function@5 from 51% to 70% (17 tasks gained, 0 lost, $p < 10^{-4}$), while a regex-forced format halves accuracy on the development split by corrupting correct answers; (iii) code-graph tools and an Evidence Buffer give no significant gain; the models route under 12% of calls to the graph tools, and the buffer adds 17–27% generated tokens. The best configuration (Qwen3.5-4B, C2) reaches 85% File@1 and 74% Function@5 at about 45 s per task, against 33% and 36% for BM25 retrieval. Gemma 4 E2B benefits from the harness in behaviour (forced answers fall from 72% to 9%) but not in accuracy, and at function level does not beat BM25. Code, per-task results and all figures are public.

1 Introduction

Coding agents built on large language models (LLMs) now resolve a substantial share of real GitHub issues [3, 7, 8]. Most of this progress relies on frontier models served through APIs. A practical first step of every issue-resolution pipeline, however, is *localization*: deciding which files and functions must change. Localization is cheaper to evaluate than repair, useful on its own (for triage, code review and developer navigation), and it is the step most likely to be feasible for small models running locally, where source code never leaves the machine.

This paper studies localization under tight constraints: models of at most 4B parameters, quantized to 4 bits, served by llama.cpp on a laptop GPU with 4 GB of memory, with no fine-tuning. Instead of changing the model, we change the *scaffolding*, the harness around it, and ask which parts of a typical agent design actually help small models. We test four hypotheses, fixed before the test runs:

H1 Each scaffolding component (harness fixes, code-graph tools, an Evidence Buffer) improves localization over a plain ReAct agent with file tools.

H2 Grammar-constrained tool calls are essential for small models: invalid tool calls are a major failure mode below 4B.

H3 The gains hold across model families (Gemma, Qwen).

H4 Fixing remains far harder than localization at this scale (planned as a secondary experiment).

Contributions.

1. A fully offline localization agent for a 4 GB GPU, with grammar-constrained decoding through llama.cpp, and an evaluation harness with paired statistics; all code and per-task results are released (Section 3).
2. A component-by-component ablation on 100 SWE-bench Lite test tasks and a 23-task development split, for two model families, with exact McNemar tests on paired outcomes (Sections 4–5).
3. Evidence that for small models the decisive components are the ones that control the *output*: grammar-constrained decoding and the answer format. Search-side additions popular for large models (graph tools, memory buffers) do not help, largely because small models do not use them (Section 6).
4. A negative result on format constraints: a regex grammar can guarantee the form of an answer while destroying its content, when the forced form diverges from the form the model is already writing (Section 4.1).

2 Related work

Issue resolution and localization. SWE-bench [3] collects real GitHub issues with their fixing patches; SWE-bench Lite is a 300-task subset of self-contained issues. Agent systems such as SWE-agent [8] and AutoCodeRover [10] interleave tool use and reasoning (in the style of ReAct [9]), while Agentless [7] shows that a fixed localize-then-repair pipeline is competitive. LocAgent [1] represents the repository as a code graph and lets an LLM agent traverse it, reporting strong file- and function-level localization. All of these use large models; we ask which of their ingredients transfer to models below 4B parameters on consumer hardware.

Constrained decoding. Grammar-constrained decoding restricts sampling to strings accepted by a grammar or JSON schema [2, 6], guaranteeing parseable tool calls. Tam et al. [5] report that strict format restrictions can reduce reasoning accuracy. Our development experiments show a related but distinct failure: a constraint that disagrees with the format the model is mid-way through writing corrupts the content of the answer (Section 4.1).

Retrieval baselines. BM25 [4] over code is the standard non-neural baseline for localization; we use it as configuration C0.

3 Setup

3.1 Task, data and metrics

Given an issue text and the repository at the issue's base commit, the agent must submit a ranked list of files and a ranked list of functions to edit. Gold locations come from the reference patch: every changed file, and the innermost function or method enclosing each changed line of an existing Python file (parsed with Python's `ast`).

Splits. We tune on the official SWE-bench Lite *dev* split (23 tasks from 6 repositories that do not appear in the test split) and evaluate once on a fixed 100-task subset of the 300-task *test* split, sampled with a fixed seed and stratified by repository (12 repositories; django 38, sympy 26, matplotlib 8, scikit-learn 8, pytest 6, sphinx 5, others $\leq$2). The subset was fixed before any test run. Every gold patch in the subset touches a single file; 12 tasks change no existing function (module-level code or new files) and are excluded from function metrics, leaving $n = 88$.

Metrics. **File@k**: every gold file is among the top-k submitted files. **Function@k**: every gold function is among the top-k submitted functions (a method also matches when the model omits its class prefix; module-style paths such as `pkg.mod` are normalised to `pkg/mod.py`). We report 95% Wilson intervals and compare configurations with exact McNemar tests on paired per-task outcomes. We also record behavioural metrics: forced submissions (the model answered only because the final turn allowed nothing else), repeated identical calls, invalid replies, empty answers, steps, generated tokens and wall time.

3.2 Models and hardware

We use Gemma 4 E2B (Google's quantization-aware Q4_0 release, about 2B effective parameters) and Qwen3.5-4B (Q4_K_M), both in non-thinking mode, served by llama.cpp (`llama-server`, build b11873, CUDA) with all layers on the GPU, flash attention, a 12,288-token context and a single slot, so the prompt prefix stays in the KV cache between steps. Decoding is greedy (temperature 0, seed 0). The machine is a laptop with an NVIDIA RTX 2050 (4 GB), 14 GB RAM and a Ryzen 5 5500H. Figure 1 shows that both models fit on the GPU: at 6K tokens of existing context, Gemma 4 E2B generates about 58 tokens/s using 2.1 GB, and Qwen3.5-4B about 31 tokens/s using up to 3.7 GB. Because generation is slow, thinking mode is disabled; every generated token costs roughly 20–30 ms.

3.3 Agent harness

Figure 2 shows the harness. Each turn the model returns one JSON object `{"thought": ..., "action": {"tool": ..., args}}`. The system prompt lists the tools; the issue is truncated to 6,000 characters; every tool output is capped at 4,000 characters (about 1K tokens); when the conversation exceeds 22,000 characters (about 6K tokens), the oldest tool outputs are replaced by one-line stubs. Episodes have 20 turns; on the last turn only `submit` is allowed, so every episode ends with an answer.

Components.

- **Grammar-constrained decoding.** The reply schema (tool name plus the tool's arguments, with bounded string patterns) is compiled by llama.cpp into a grammar that holds from the

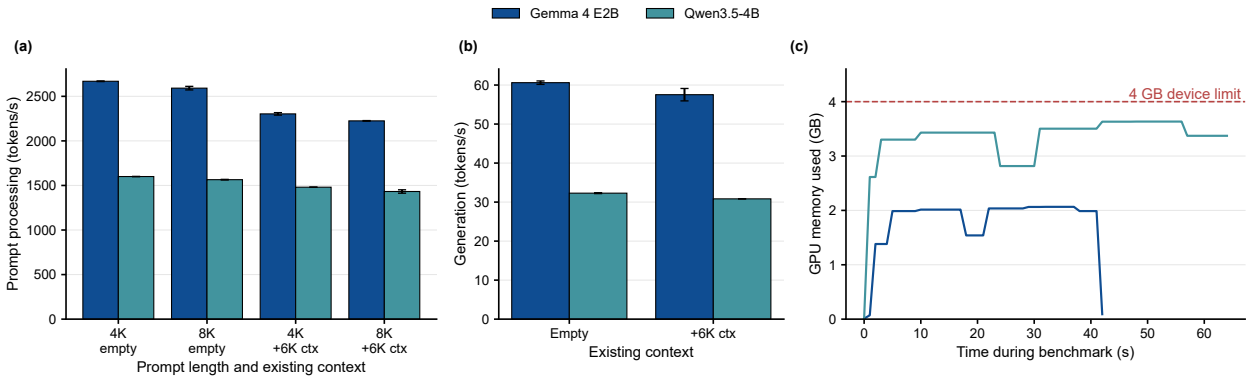


Figure 1: Hardware benchmark (`llama-bench`, 2 repetitions, error bars: range). (a) Prompt processing and (b) generation throughput on an empty context and after 6K tokens of context. (c) GPU memory during the benchmark; both models stay under the 4 GB device limit.

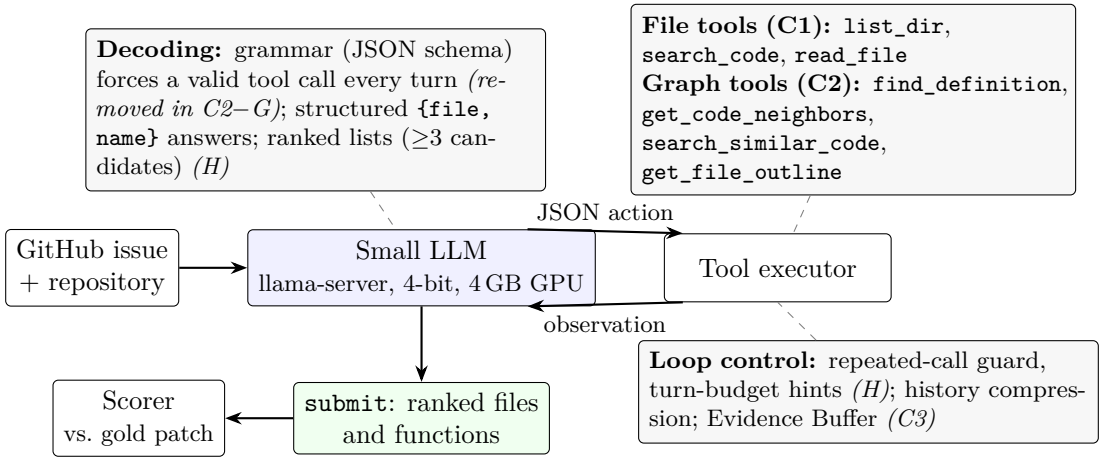


Figure 2: Agent harness. The model alternates tool calls and observations for up to 20 turns, then submits ranked files and functions. Components are switched on cumulatively by the configurations in Table 1.

first token. Requests go through `/apply-template` and `/completion`: the chat endpoint applied the grammar only after a thinking block, which Gemma 4 E2B opened on its own, producing invalid replies.

- **Harness fixes (H)**, four changes found on the dev split (Section 4): a *structured answer* in which each function is an object `{"file", "name"}`; a *repeated-call guard* that refuses an exact repeat of an earlier call and tells the model to act differently or submit; *budget hints* (`[Turn` i `of 20]`, and in the last five turns "submit as soon as you have a candidate"); and *ranked lists* (`submit` requires 3–5 files and 3–10 functions, best first).
- **Graph tools (C2)** over an `ast` index of the repository: definition lookup, a name-matched call graph (callers and callees), BM25 search over function bodies, and file outlines.
- **Evidence Buffer (C3)**: each reply carries a required `evidence` field (at most 200 characters); collected facts are appended to the latest observation, and old outputs are hidden in one chunk so that the prompt prefix, and thus the KV cache, changes less often.

Table 1: Configurations. Each row adds one component to the row above; C2−G removes grammar-constrained decoding from C2.

ID	Configuration	Notes
C0	BM25 retrieval, no LLM	files ranked by issue–file BM25; functions ranked within the top-5 files
C1	ReAct agent + file tools	free-form function names, no guards
C1+H	C1 + harness fixes	structured answers, repeat guard, budget hints, ranked lists
C2	C1+H + graph tools	best configuration on the dev split ("full system")
C3	C2 + Evidence Buffer	
C2−G	C2 without grammar	replies parsed from free text; parse failures count as invalid steps

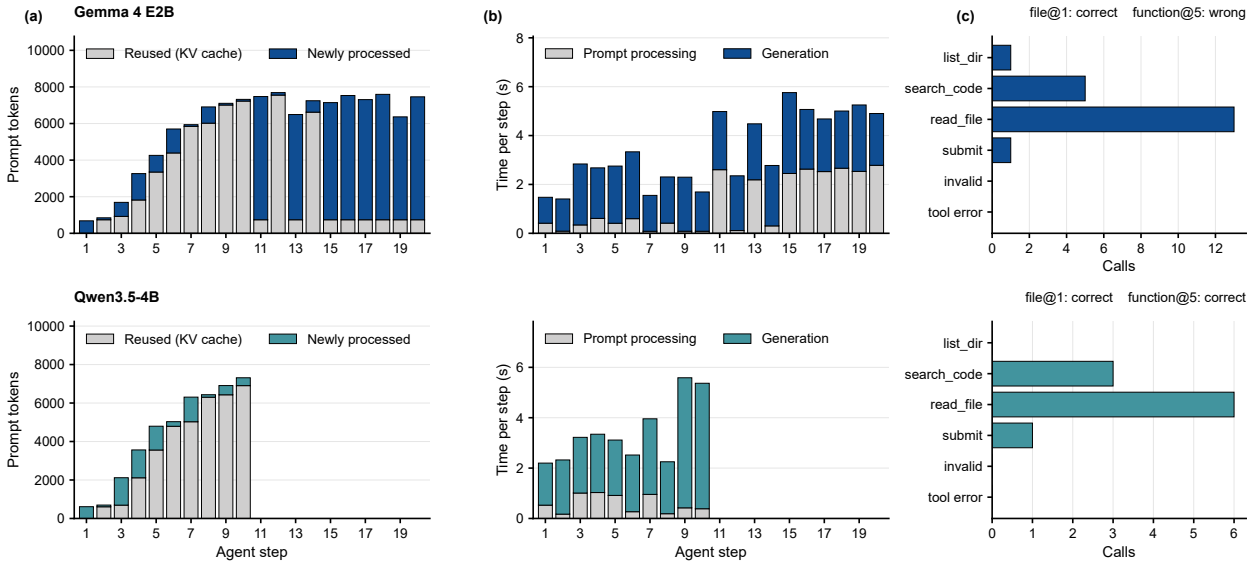


Figure 3: One episode per model (task `astropy-12907`, configuration C1). (a) Prompt tokens per step, split into reused (KV cache) and newly processed; once history compression starts (step 11 for Gemma 4 E2B), the cache is lost. (b) Time per step. (c) Tool calls. Gemma 4 E2B uses all 20 steps and names a wrong function; Qwen3.5-4B answers correctly in 10.

4 Development experiments

All harness decisions were made on the 23-task dev split, in the order below; Table 2 summarises them, and Appendix B has the per-experiment figures.

Smoke test. A single task end to end (Figure 3) exposed two engineering problems that shaped the harness: the chat endpoint's lazy grammar (fixed by `/completion`, giving 0 invalid replies), and the cost of history compression, which changes the prompt prefix so that later steps reprocess about 6K tokens instead of reusing the KV cache.

4.1 Answer format: form versus content

With C1, both models reach 83% File@1, but function accuracy is 20–40 points lower, and 43% of Qwen3.5-4B's answers write functions in a module style (`pydicom.dataset.Dataset.to_json_dict`) instead of the requested `path.py::Name`. Forcing the requested format with a regex in the grammar (experiment 04) removed every malformed name, yet halved Qwen3.5-4B's Function@5 from 62% to

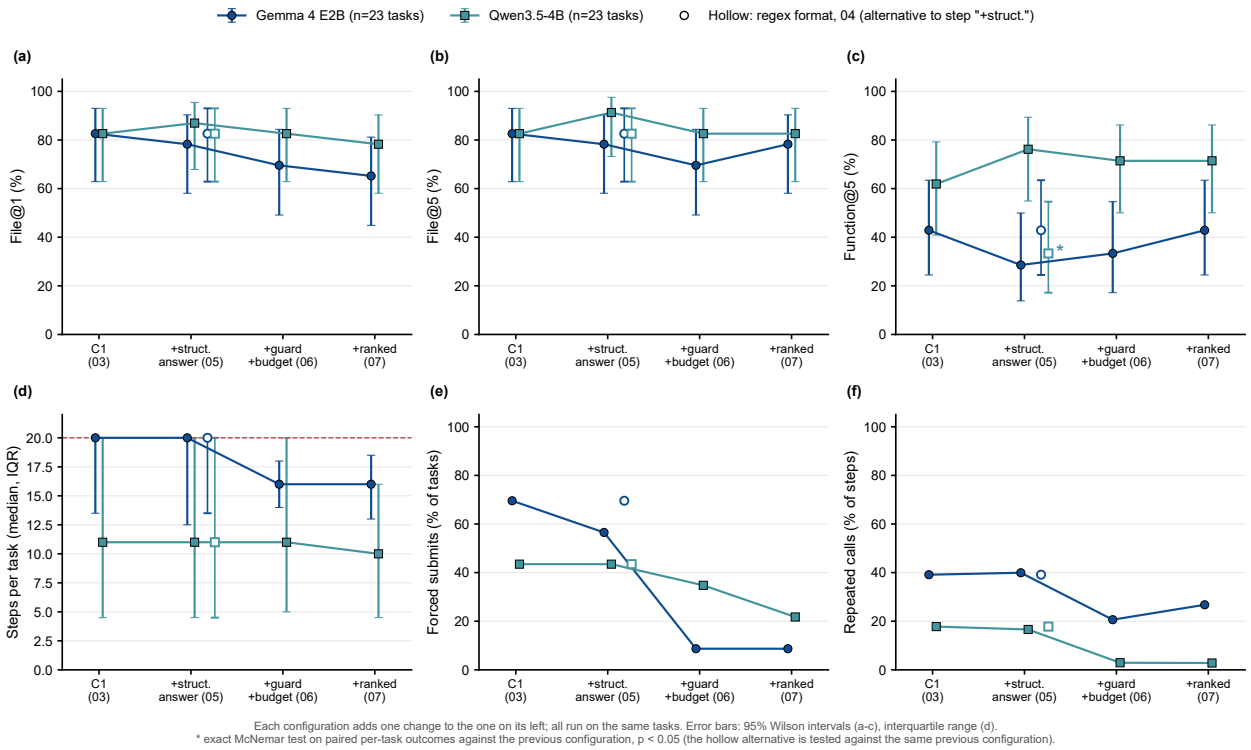


Figure 4: Dev-split ablation ladder (23 tasks; 21 with gold functions). Each step adds one change; the hollow marker is the regex-format alternative (experiment 04). Error bars: 95% Wilson intervals; $*$: exact McNemar $p < 0.05$ against the previous step.

33% (6 tasks lost, none gained, $p = 0.031$, the only significant change on the dev split). The model begins its preferred dotted form, and the grammar does not let the string end without `.py::`, so the model fills it with junk until the length bound forces the suffix:

```
pvlib.iam.physical.physical_1_interface_2_interface_3_interface_.._.py::physical
```

A *structured* format, which gives the file and the name separate fields, recovers these answers (Qwen3.5-4B Function@5 76%), but hurts Gemma 4 E2B (43% → 29%), which leaves the function list empty in two tasks. The repeated-call guard and budget hints then cut Gemma 4 E2B's forced submissions from 57% to 9% without changing accuracy, and ranked lists let Gemma 4 E2B's second and third guesses count (Function@5 33% → 43%). Figure 4 shows the ladder.

Graph tools and Evidence Buffer on dev. Graph tools gave Gemma 4 E2B its best dev accuracy (Function@5 52%) and left Qwen3.5-4B unchanged, but neither change was significant ($p \geq 0.63$), and fewer than 10% of calls went to the new tools. The Evidence Buffer gave no gain and cost 43–52% more time per task. We therefore fixed C2 as the full system, kept C3 as an ablation row, and ran all configurations on the test set.

5 Test-set results

Table 3 gives the main results on the 100 test tasks, and Figure 5 the ablation ladder with paired tests.

Table 2: Dev split (23 tasks, 6 repositories; Function@5 over 21 tasks). Percentages; experiment numbers refer to the result folders in the repository. C3−G (dev only) is C3 without grammar.

		Gemma 4 E2B			Qwen3.5-4B		
Exp.	Configuration	File@1	File@5	Func@5	File@1	File@5	Func@5
09	C0: BM25	35	57	43	35	57	43
03	C1: file tools	83	83	43	83	83	62
04	+ regex-forced names	83	83	43	83	83	**33**
05	+ structured answers	78	78	29	87	91	**76**
06	+ repeat guard, budget hints	70	70	33	83	83	71
07	+ ranked lists (= C1+H)	65	78	43	78	83	71
10	C2: + graph tools	74	83	52	78	87	71
11	C3: + Evidence Buffer	78	83	43	78	87	67
12	C3−G: C3 without grammar	70	74	38	57	61	57

Table 3: Main results on the SWE-bench Lite test subset (100 tasks; Function@5 over 88). Percentages with 95% Wilson intervals; last two columns: median wall time per task in seconds (Gemma 4 E2B / Qwen3.5-4B).

		Gemma 4 E2B			**Qwen3.5-4B**			Time (s)	
ID	Configuration	File@1	File@5	Func@5	File@1	File@5	Func@5	G	Q
C0	BM25 retrieval, no LLM	33 [25,43]	64 [54,73]	36 [27,47]	33 [25,43]	64 [54,73]	36 [27,47]	2	
C1	ReAct + file tools	69 [59,77]	69 [59,77]	35 [26,46]	81 [72,87]	82 [73,88]	51 [41,61]	55	41
C1+H	+ harness fixes (H)	65 [55,74]	75 [66,82]	40 [30,50]	80 [71,87]	90 [83,94]	70 [60,79]	42	48
C2	+ graph tools	66 [56,75]	75 [66,82]	36 [27,47]	85 [77,91]	89 [81,94]	74 [64,82]	43	40
C3	+ Evidence Buffer	62 [52,71]	71 [61,79]	35 [26,46]	80 [71,87]	88 [80,93]	65 [54,74]	52	46
C2−G	C2 without grammar	56 [46,65]	57 [47,66]	26 [18,36]	39 [30,49]	39 [30,49]	28 [20,39]	41	64

5.1 Grammar-constrained decoding is essential (H2 supported)

Compared with C2, removing the grammar drops Qwen3.5-4B's File@1 from 85% to 39% (0 tasks gained, 46 lost, $p = 3 \times 10^{-14}$) and its Function@5 from 74% to 28% (1 gained, 41 lost, $p = 2 \times 10^{-11}$). Gemma 4 E2B loses 18 points of File@5 (1 gained, 19 lost, $p = 4 \times 10^{-5}$); its drops in File@1 (66% → 56%, $p = 0.052$) and Function@5 (36% → 26%, $p = 0.11$) are not significant at $n = 100$. Figure 6 breaks down why. Free-text replies are invalid in 19% (Gemma 4 E2B) and 23% (Qwen3.5-4B) of steps, and at least one invalid reply occurs in 45% and 79% of episodes. Gemma 4 E2B's failures are all malformed JSON (unescaped characters, missing delimiters in long thoughts). Qwen3.5-4B's are mostly structural: in 173 steps it wrote a complete answer but omitted `"tool": "submit"`, putting the files directly into the action, and in 119 it named a tool that does not exist. Because its final answer is the step it most often writes wrongly, 59% of Qwen3.5-4B's episodes end with an empty answer, against 0% with the grammar. A more forgiving parser would recover some of these answers, so the size of the drop depends partly on our parser; the grammar removes the need for such repair altogether. The episodes also become longer: with the grammar, Qwen3.5-4B finishes half of its episodes within 12 steps, without it most run to the limit (Figure 6c).

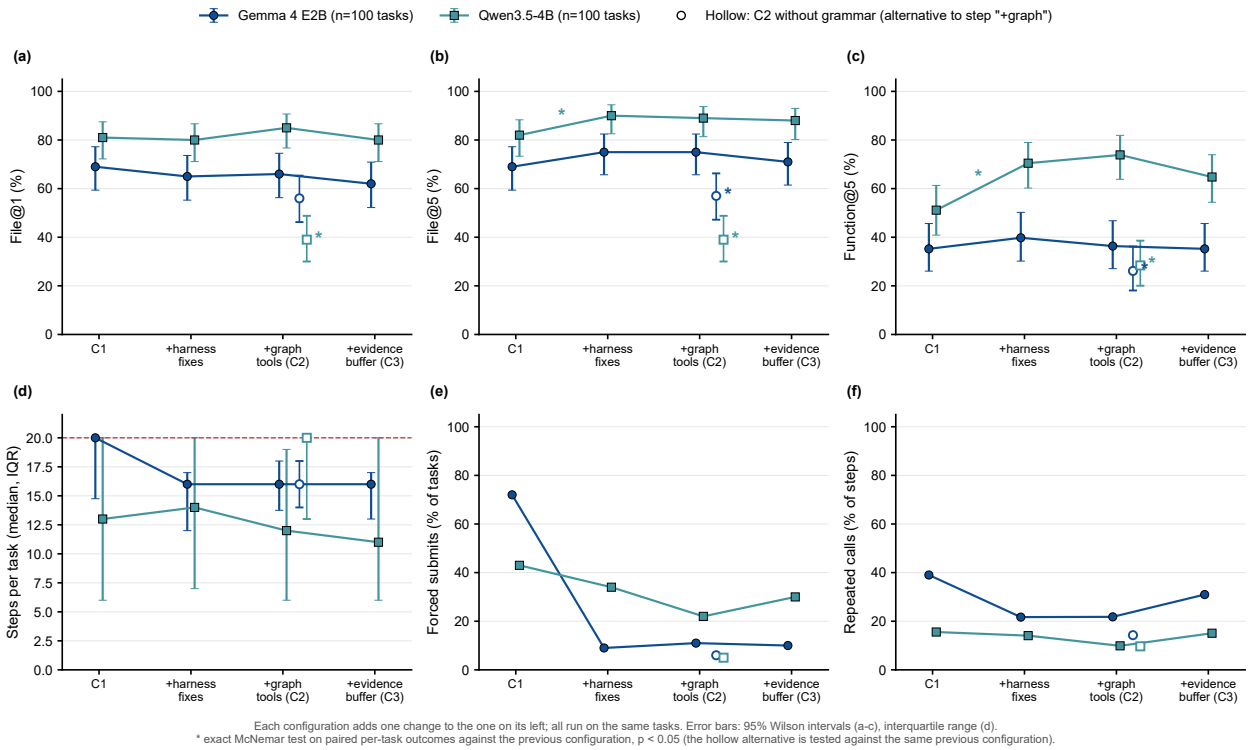


Figure 5: Test-set ablation ladder (100 tasks; 88 with gold functions). (a–c) Accuracy; (d) steps per task (median, interquartile range; dashed line: 20-step limit); (e) episodes that ended only because the last turn forced `submit`; (f) share of steps repeating an earlier call. Hollow markers: C2 without grammar. Error bars: 95% Wilson intervals; ∗: exact McNemar $p < 0.05$ against the previous configuration (the hollow marker is tested against C1+H; the text compares it with C2).

5.2 Harness fixes help Qwen3.5-4B; graph tools and the Evidence Buffer do not (H1 mostly rejected)

Harness fixes. C1→C1+H raises Qwen3.5-4B's Function@5 from 51% to 70% (17 tasks gained, *none* lost, $p = 1.5 \times 10^{-5}$) and File@5 from 82% to 90% ($p = 0.039$). Malformed function names fall from 44% of Qwen3.5-4B's tasks to 1%: the structured answer converts the model's existing knowledge into scorable answers. For Gemma 4 E2B, the same fixes change behaviour (forced submissions 72% → 9%, repeated calls 39% → 22% of steps, mean time per task 51 s → 40 s) but none of its accuracy changes is significant.

Graph tools. C1+H→C2 changes no metric significantly for either model (Qwen3.5-4B File@1 +5 tasks, 0 lost, $p = 0.063$). Figure 7 shows why the effect is small: graph tools receive 11% of Gemma 4 E2B's and 9% of Qwen3.5-4B's calls, almost all of them `find_definition`; in C2, Qwen3.5-4B calls `get_code_neighbors` and `get_file_outline` once each and never calls `search_similar_code`. Both models keep spending most turns on `search_code` and `read_file`.

Evidence Buffer. C2→C3 lowers every accuracy metric for both models (none significantly; Qwen3.5-4B Function@5 74% → 65%, 5 gained, 13 lost, $p = 0.096$), while generating 27% (Gemma 4 E2B) and 17% (Qwen3.5-4B) more tokens and taking 23% and 14% longer per task (Figure 8). Within a 20-turn, ∼6K-token history, little context is lost, so the buffer pays its cost without a

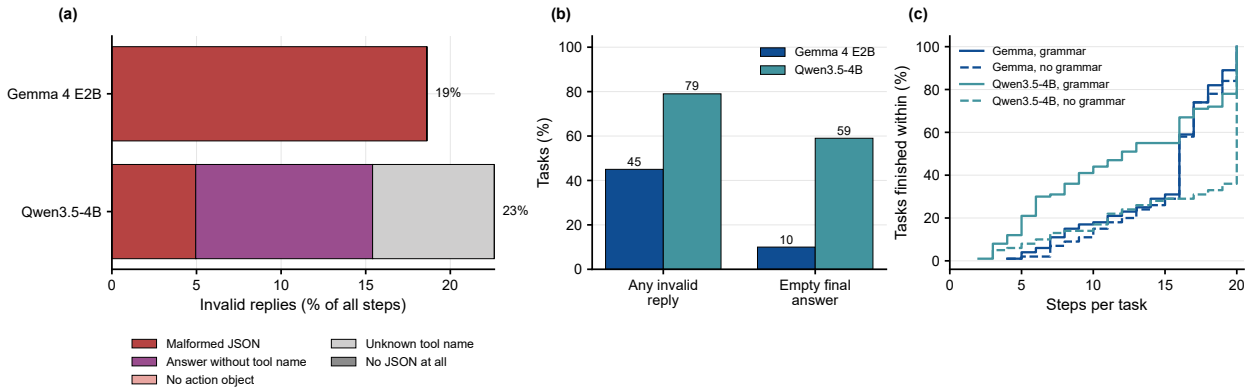


Figure 6: What goes wrong without grammar-constrained decoding (C2−G, 100 tasks). (a) Invalid replies as a share of all steps, by type; "answer without tool name" is a well-formed final answer missing `"tool": "submit"`. (b) Episodes with at least one invalid reply and with an empty final answer. (c) Cumulative distribution of episode length with (solid) and without (dashed) the grammar.

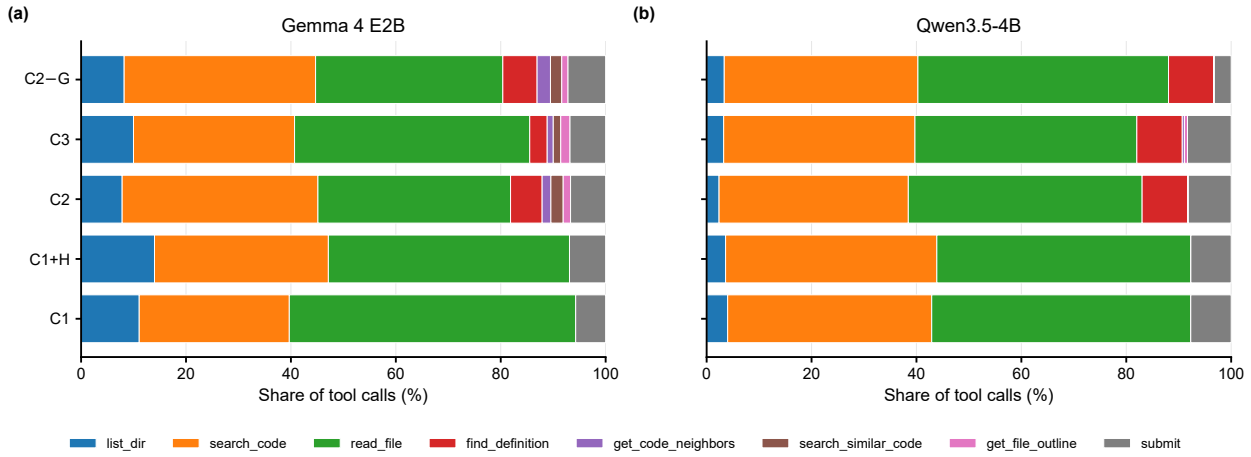


Figure 7: Tool usage by configuration (share of all tool calls, 100 tasks). Graph tools (red, purple, brown, pink) are available from C2 on but receive about one call in ten; `find_definition` accounts for most of them.

problem to solve.

5.3 Model families differ (H3 partly supported)

Qwen3.5-4B is the stronger localizer in every configuration (Table 3), by 12–19 points of File@1 and 16–38 points of Function@5 with the grammar on. The two models also respond differently to the same change: the structured answer format and ranked lists lift Qwen3.5-4B and do little for Gemma 4 E2B, and on the dev split the structured format hurt Gemma 4 E2B. What does transfer across families is the effect of the grammar, and the absence of a gain from graph tools and the Evidence Buffer.

5.4 Comparison with retrieval

BM25 (C0) reaches 33% File@1, 64% File@5 and 36% Function@5 (47% at Function@10) in about 2 s per task on the CPU. The agents' advantage is concentrated in the top file: Gemma 4 E2B C2

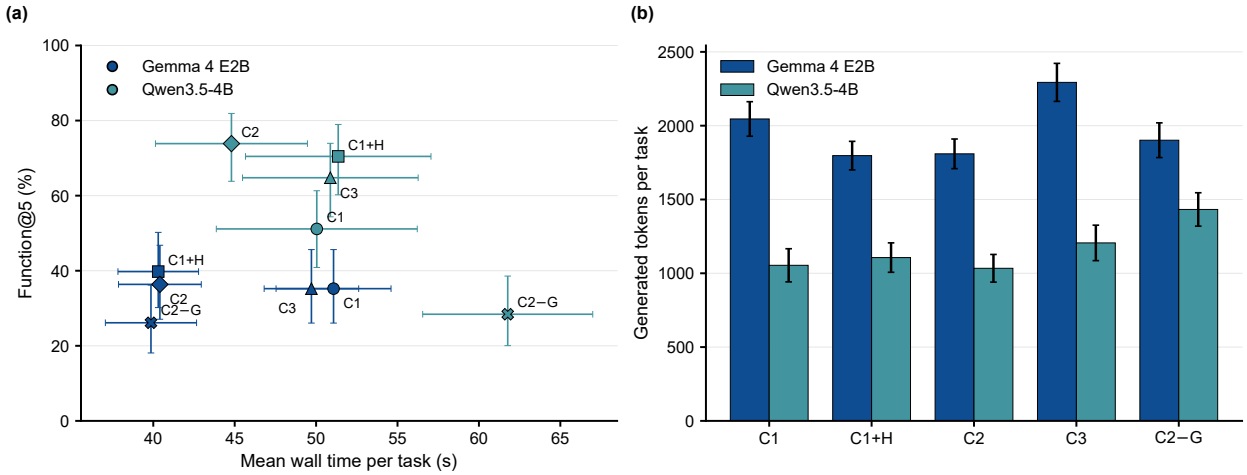


Figure 8: Cost and accuracy. (a) Function@5 against mean wall time per task (error bars: 95% Wilson interval and 95% CI of the mean). (b) Generated tokens per task (mean, 95% CI). On a 4 GB GPU, generation dominates time, so components that make the model write more (Evidence Buffer, free-text replies) are expensive.

gains 41 tasks of File@1 over BM25 and loses 8 ($p = 2 \times 10^{-6}$), Qwen3.5-4B gains 55 and loses 3. At function level, Qwen3.5-4B C2 beats BM25 clearly (74% vs. 36%, 37 gained, 4 lost, $p = 10^{-7}$), but Gemma 4 E2B C2 ties it (36% vs. 36%, 14 gained, 14 lost). Figure 9 shows the same pattern per repository: the agents dominate BM25 on file localization almost everywhere, while Gemma 4 E2B's function-level accuracy stays near BM25's on the two largest repositories (django, sympy).

5.5 Ranked lists, complementarity and headroom

Ranked lists add real but modest value: when the correct file is not first, it is second or third in 9–10% of tasks for Gemma 4 E2B and 4–10% for Qwen3.5-4B (Figure 10). Both models submit exactly the minimum of three candidates, so the lists act as a floor rather than as calibrated rankings.

The two models' successes are nearly nested (Figure 11a): among 88 tasks, Qwen3.5-4B C2 solves 37 that Gemma 4 E2B C2 misses, while Gemma 4 E2B solves only 4 that Qwen3.5-4B misses; 19 tasks defeat both. Across configurations, however, there is headroom: a task is solved by *some* configuration of Gemma 4 E2B in 62% of cases (best single configuration: 40%) and of Qwen3.5-4B in 82% (best: 74%); any configuration of either model covers 85%, and adding BM25 only reaches 86% (Figure 11b). Greedy decoding is deterministic, so this spread comes from the configurations, not from sampling: small changes to the prompt or tools change which tasks succeed, which suggests that ensembling or self-consistency across configurations could close part of the gap.

5.6 Does the dev split predict the test split?

Figure 12 compares the same configurations on both splits. For Qwen3.5-4B the dev split is a good guide (all Function@5 points within 11 points of the diagonal). For Gemma 4 E2B it is optimistic: File@1 drops by 8–16 points from dev to test in three of four configurations, and C2's dev advantage (Function@5 52%) disappears on the test set (36%), which is consistent with the wide dev intervals ($n = 21$). With 23 tasks, only changes of about six tasks in one direction are detectable; dev results should guide engineering, not support claims.

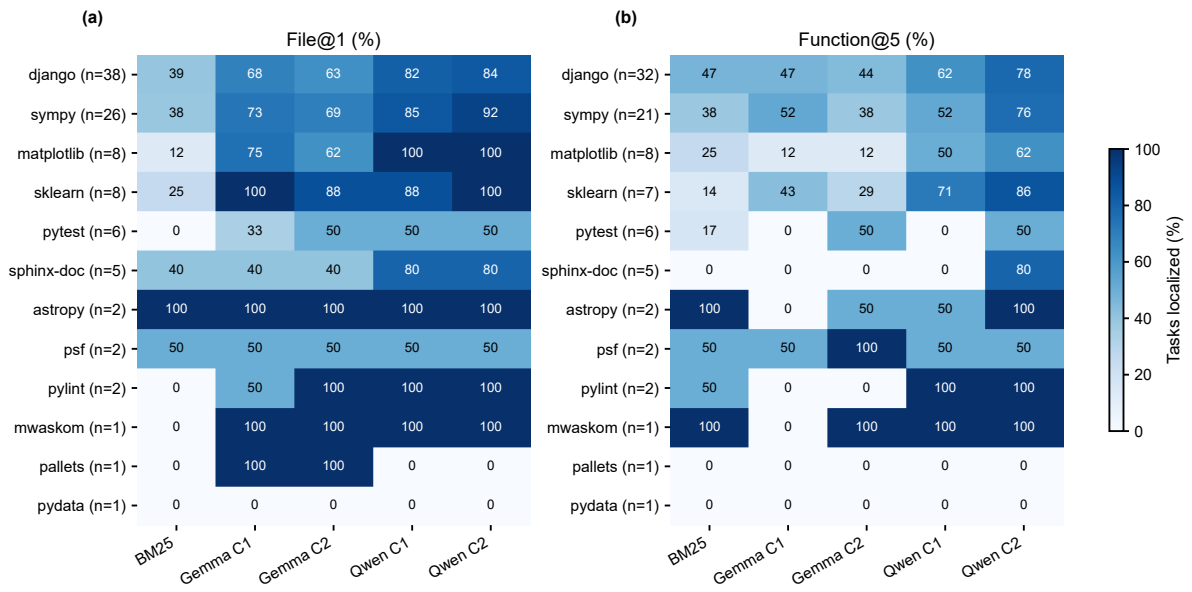


Figure 9: Accuracy per repository on the test subset (n = tasks per repository; function metrics exclude tasks without gold functions). Repositories with one or two tasks are shown for completeness, not for comparison.

6 Discussion

Control the output, not the search. The components that helped are those that control what the model emits: the grammar that makes every turn a valid call, the structured answer that separates file from function, and the guards that make the model stop. The components that extend the search (graph tools, an evidence memory) did not help, and the tool-usage data explain why: small models largely ignore tools they were not trained to rely on and fall back to grep-and-read. Tool *availability* is not tool *adoption*. For small models, graph information may have to be pushed into the prompt (for example, showing definitions of identifiers that appear in the issue) instead of offered as a tool to pull.

Constraints must follow the model. The regex experiment and the free-text failures are two sides of the same issue. Unconstrained, Qwen3.5-4B writes sensible answers in a slightly wrong shape; constrained in a way that disagrees with its preferred shape, it writes junk to satisfy the constraint. A constraint works when it leaves room for what the model is already trying to write (separate fields, permissive names), and fails when it forces a token sequence the model has not planned.

Behaviour fixes are not accuracy fixes. Gemma 4 E2B's looping (72% forced submissions in C1) looked like the cause of its errors, but stopping it saved 21% of time and changed accuracy by nothing measurable: it looped because it did not know where to look. Behavioural metrics are useful for cost but are poor proxies for accuracy.

Hypotheses. H1 is supported only for the harness fixes, and only for Qwen3.5-4B; graph tools and the Evidence Buffer do not help. H2 is supported for both small models; we could not test whether the grammar matters less for large models (Section 8). H3 is partly supported: the grammar effect

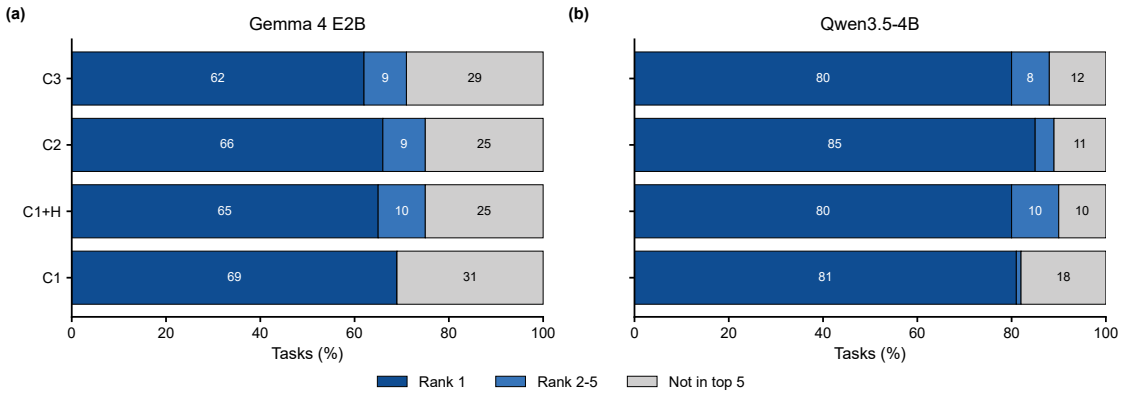


Figure 10: Rank of the first correct file in the submitted list. C1 asks for up to five files but the models submit about one; from C1+H on, at least three are required. No model ever submits more than three files, so ranks 4–5 are empty.

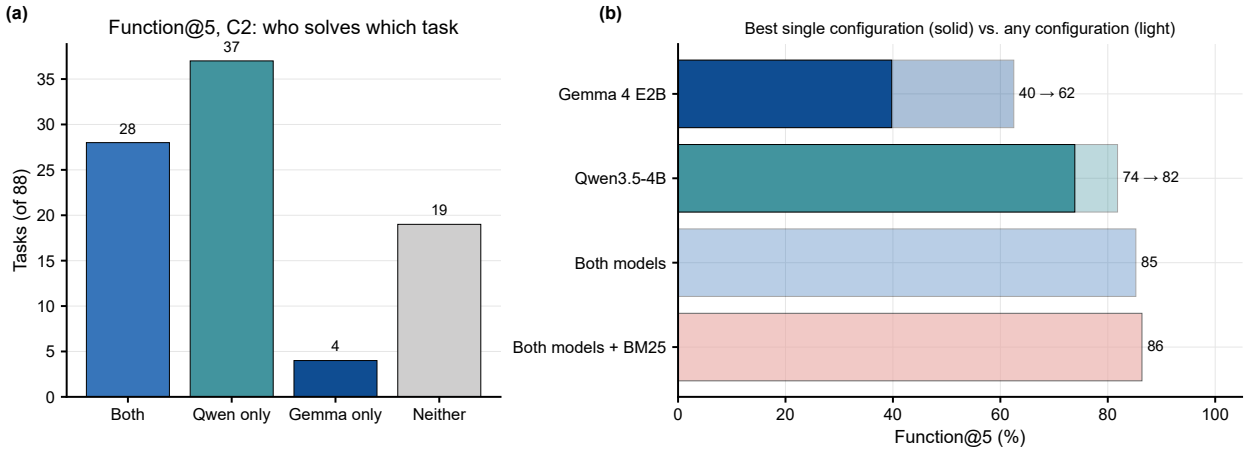


Figure 11: Complementarity and headroom (Function@5, 88 tasks). (a) Which model solves which task with configuration C2. (b) Best single configuration (solid) against the share of tasks solved by at least one configuration (light), per model, for both models together, and with BM25 added.

transfers across families, the effect of answer-format changes does not. H4 was not tested: the repair experiment was not run within this study.

7 Limitations

- **Sample size.** 100 test tasks (88 with gold functions) detect differences of roughly 10–15 points between paired configurations; smaller effects, such as those of graph tools, remain unresolved. The dev split has only 23 tasks.
- **One run per configuration.** Decoding is greedy with a fixed seed, so runs are deterministic, but results may be sensitive to prompt wording, which we did not vary systematically.
- **Two models, one quantization each.** Results may differ for other families (e.g., Phi, Llama) and for higher-precision weights.
- **Parser dependence.** The C2−G drop includes failures a more forgiving parser could repair

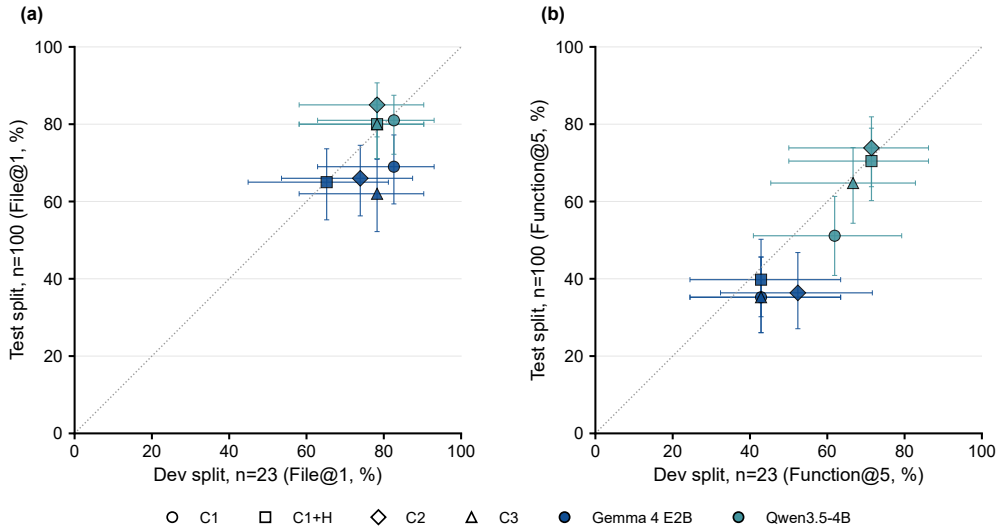


Figure 12: Dev versus test accuracy for the same configurations (error bars: 95% Wilson intervals; dotted: equality). Dev results overestimate Gemma 4 E2B's test accuracy.

(Section 5); it measures our harness without the grammar, not the best possible free-text harness.

- **Gold locations.** Gold functions are derived from the reference patch; other valid fixes may edit different functions. All gold patches in the subset touch one file.
- **No large-model reference and no repair experiment** (Section 8).

8 Future work

- **Large-model reference.** We planned to run C2 with Gemma 4 31B through the free Gemini API on 50 test tasks (about 600–750 requests), to test whether the grammar and harness matter less at scale (H2). Under the free-tier request limits available to us, the run would have taken roughly a week of continuous scheduling, so it is left for future work.
- **Repair (H4).** Run the best configuration end to end, generating and testing patches on a subset, to measure how far a 4 GB GPU gets on fixing.
- **Ensembles over configurations.** The 85% union across configurations (Figure 11) suggests voting or self-consistency across prompts and tool sets.
- **Pushed graph context.** Inject graph lookups automatically instead of exposing them as tools.
- **Scale.** The full 300-task test split, a third model family, and longer budgets, where an Evidence Buffer may start to pay off.

9 Conclusion

Off-the-shelf models of 2–4B parameters, running offline on a 4 GB laptop GPU without training, can localize the right file for 85% and the right function for 74% of SWE-bench Lite test issues in under

a minute each, with the right scaffolding. The right scaffolding, however, is not the search-heavy design that works for large models. It is the part that controls the output: grammar-constrained decoding, an answer format that matches how the model already writes, and simple guards. For small models, getting the answer out matters as much as finding it.

Reproducibility. The repository contains the harness (`agent/`), run scripts (`scripts/`), every experiment's configuration, per-task results and figures (`results/`), and the code that regenerates every figure and table in this paper from those results (`analysis/`, `paper/`). Full trajectories are kept outside the repository because of their size.

References

[1] Z. Chen, X. Tang, G. Deng, F. Wu, J. Wu, Z. Jiang, V. Prasanna, A. Cohan, and X. Wang. LocAgent: Graph-guided LLM agents for code localization. In *Proceedings of ACL*, 2025. arXiv:2503.09089.

[2] S. Geng, M. Josifoski, M. Peyrard, and R. West. Grammar-constrained decoding for structured NLP tasks without finetuning. In *Proceedings of EMNLP*, 2023. arXiv:2305.13971.

[3] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of ICLR*, 2024. arXiv:2310.06770.

[4] S. Robertson and H. Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[5] Z. R. Tam, C.-K. Wu, Y.-L. Tsai, C.-Y. Lin, H.-y. Lee, and Y.-N. Chen. Let me speak freely? A study on the impact of format restrictions on performance of large language models. In *Proceedings of EMNLP: Industry Track*, 2024. arXiv:2408.02442.

[6] B. T. Willard and R. Louf. Efficient guided generation for large language models. arXiv:2307.09702, 2023.

[7] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang. Agentless: Demystifying LLM-based software engineering agents. arXiv:2407.01489, 2024.

[8] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in NeurIPS*, 2024. arXiv:2405.15793.

[9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *Proceedings of ICLR*, 2023. arXiv:2210.03629.

[10] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury. AutoCodeRover: Autonomous program improvement. In *Proceedings of ISSTA*, 2024. arXiv:2404.05427.

A Run details

Table 4: Run parameters shared by all agent configurations.

Parameter	Value
Runtime	llama.cpp `llama-server` b11873, CUDA, all layers on GPU, flash attention
Models	Gemma 4 E2B (QAT Q4_0, 3.1 GB file); Qwen3.5-4B (Q4_K_M, 2.6 GB file)
Context	12,288 tokens, one slot, non-thinking mode
Decoding	greedy (temperature 0, seed 0); grammar from the reply JSON schema (except C2−G)
Episode	20 turns; last turn allows only `submit`
Prompt budget	issue $\leq$ 6,000 chars; tool output $\leq$ 4,000 chars; history $\leq$ 22,000 chars
Data	SWE-bench Lite dev (23 tasks) and a seeded, repository-stratified 100-task test subset
Compute	1,000 test episodes, 13.3 h total wall time on one RTX 2050

System prompt.

```
You are a software engineer finding the location of a bug in a Python repository.
Read the GitHub issue, explore the code with the tools, and submit the files and
functions that must be edited to fix the issue.  Do not write a fix.
Tools:  {tool descriptions}
Reply with exactly one JSON object per turn:  {"thought":  "<one or two sentences>",
"action":  {"tool":  "<tool name>", ...arguments}}
You have 20 turns.  Submit as soon as you are confident.
```

Experiment index. Every experiment lives in `results/<date>_<NN>_<name>/` with its configuration, per-task results, metrics, figures and (for dev experiments) notes: 01 hardware benchmark; 02 end-to-end smoke test; 03 C1 (dev); 04 regex function names (dev); 05 structured answers (dev); 06 repeat guard and budget hints (dev); 07 ranked lists (dev); 08 dev ablation; 09 C0 BM25 (dev); 10 C2 (dev); 11 C3 (dev); 12 C3−G (dev); 13 C0 (test); 14 C2 (test); 15 C1 (test); 16 C1+H (test); 17 C3 (test); 18 C2−G (test); 19 test ablation.

B Per-experiment figures

Each experiment's summary figure shows (a) accuracy with 95% Wilson intervals, (b) agent steps and (c) wall time per task, (d) failure modes and, from experiment 07 on, (e) tool usage. The outcome figures mark, for every task, whether the model found the file only, the file and the function, or neither (top-5).

B.1 Development split

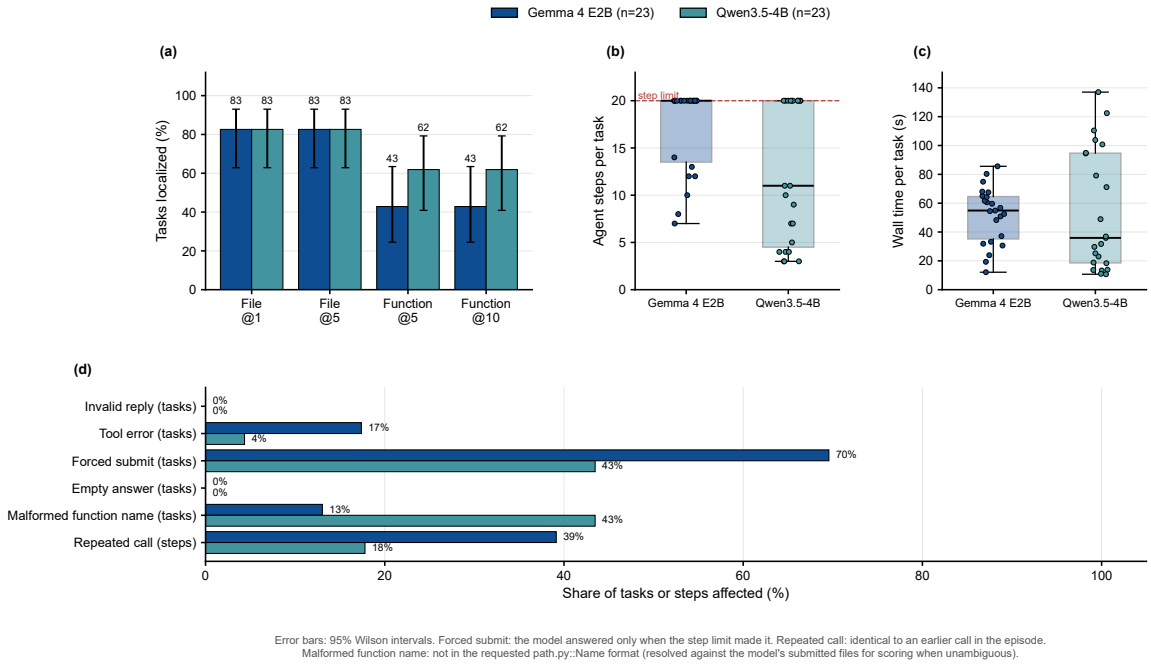
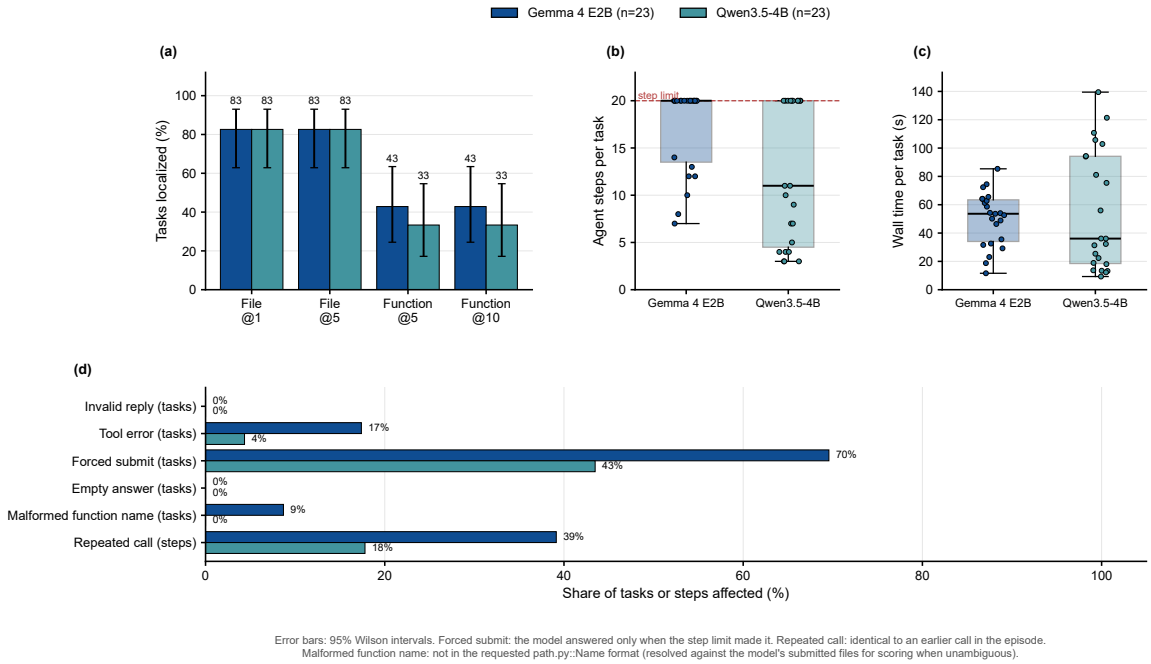
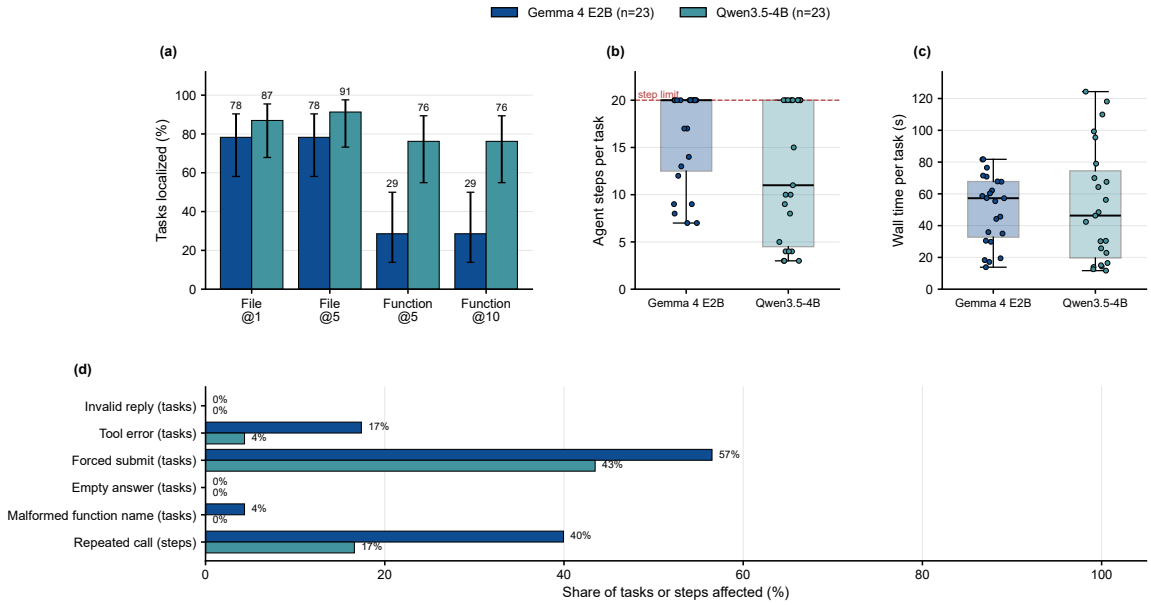


Figure 13: Exp. 03, C1 on dev. File tools only. Both models find the file in 83% of tasks; function accuracy is lower, Gemma 4 E2B hits the step limit in 70% of episodes, and 43% of Qwen3.5-4B's tasks contain malformed function names.



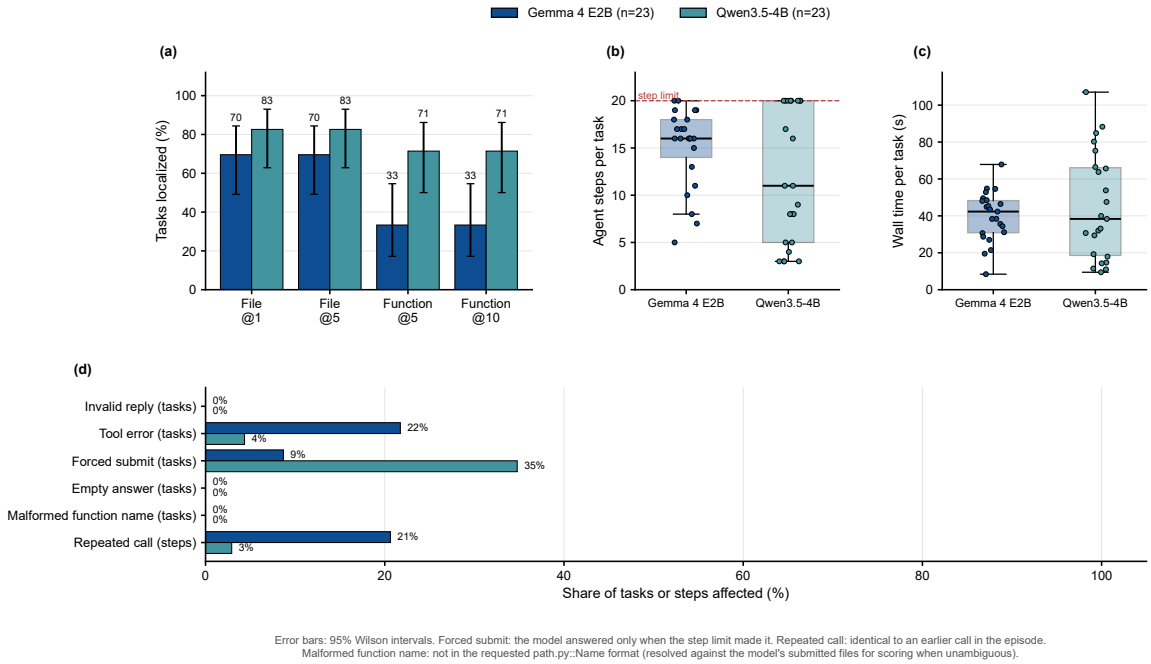
Error bars: 95% Wilson intervals. Forced submit: the model answered only when the step limit made it. Repeated call: identical to an earlier call in the episode.
Malformed function name: not in the requested path.py::Name format (resolved against the model's submitted files for scoring when unambiguous).

Figure 14: Exp. 04, regex-forced function names (dev). Malformed names vanish, but Qwen3.5-4B's Function@5 falls from 62% to 33%: the regex corrupts answers the model was writing in its own format.



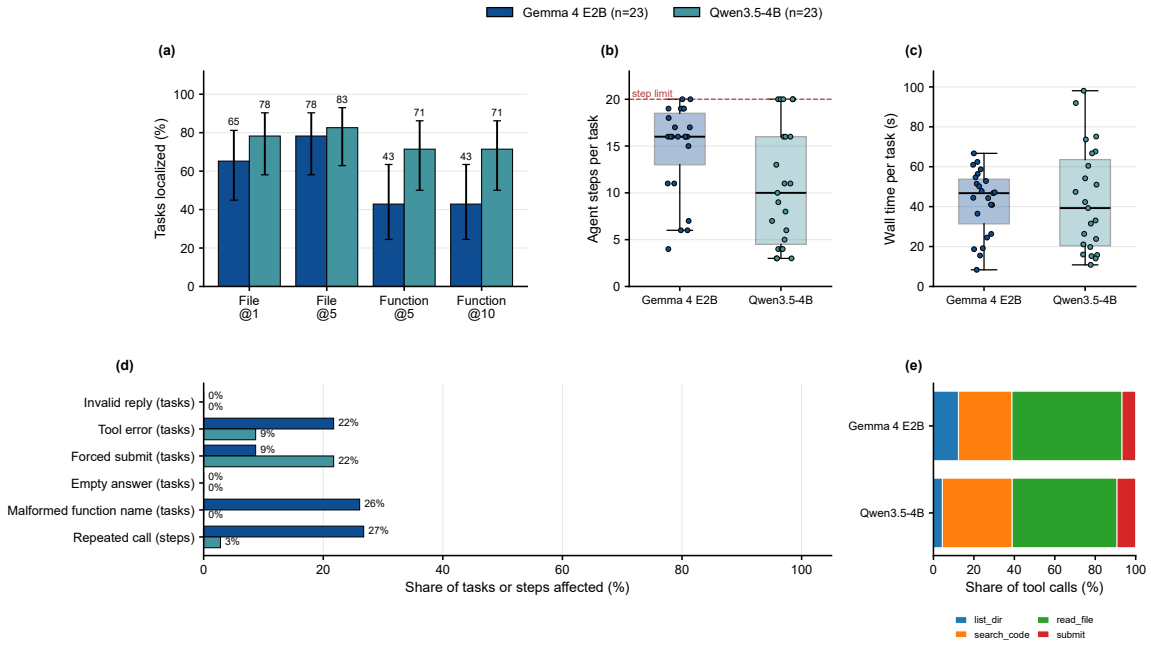
Error bars: 95% Wilson intervals. Forced submit: the model answered only when the step limit made it. Repeated call: identical to an earlier call in the episode.
Malformed function name: not in the requested path.py::Name format (resolved against the model's submitted files for scoring when unambiguous).

Figure 15: Exp. 05, structured answers (dev). Qwen3.5-4B Function@5 rises to 76%; Gemma 4 E2B falls to 29%, leaving function lists empty in two tasks.



Error bars: 95% Wilson intervals. Forced submit: the model answered only when the step limit made it. Repeated call: identical to an earlier call in the episode.
Malformed function name: not in the requested path.py::Name format (resolved against the model's submitted files for scoring when unambiguous).

Figure 16: Exp. 06, repeat guard and budget hints (dev). Gemma 4 E2B's forced submissions fall from 57% to 9% and its time per task by 26%, with no accuracy change.



Error bars: 95% Wilson intervals. Forced submit: the model answered only when the step limit made it. Repeated call: identical to an earlier call in the episode.
Malformed function name: not in the requested path.py::Name format (resolved against the model's submitted files for scoring when unambiguous).

Figure 17: Exp. 07, ranked lists (dev). Both models submit exactly three candidates; Gemma 4 E2B's Function@5 rises to 43%, top-1 accuracy drops slightly for both.

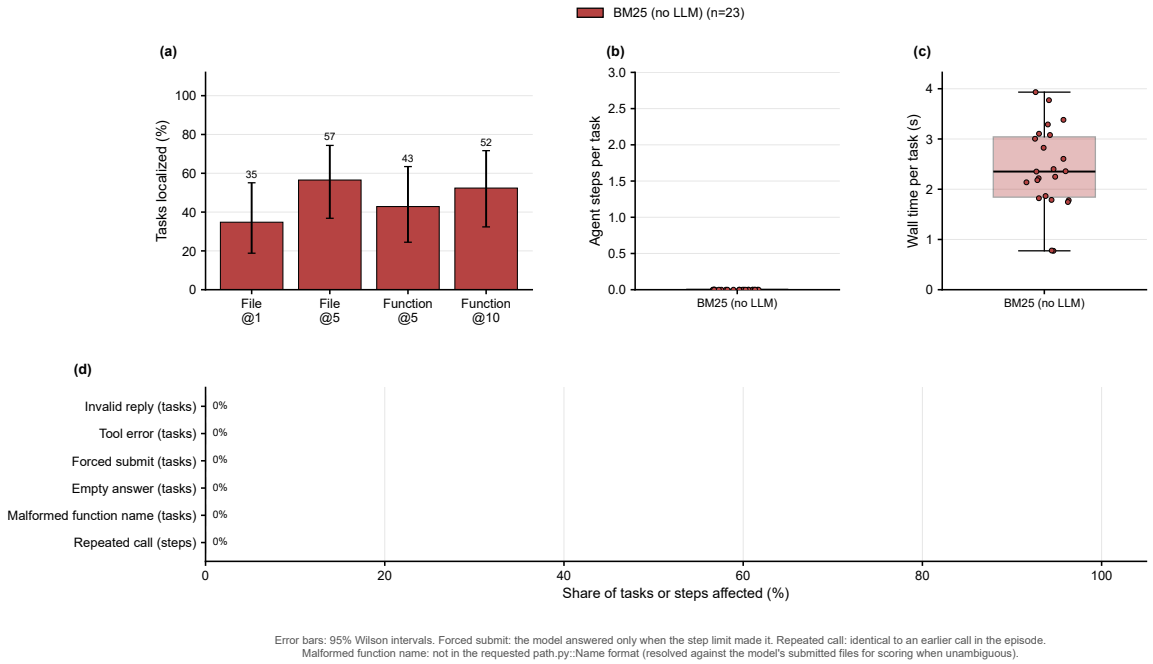


Figure 18: Exp. 09, C0 BM25 retrieval (dev). 35% File@1, 43% Function@5, about 2 s per task on the CPU.

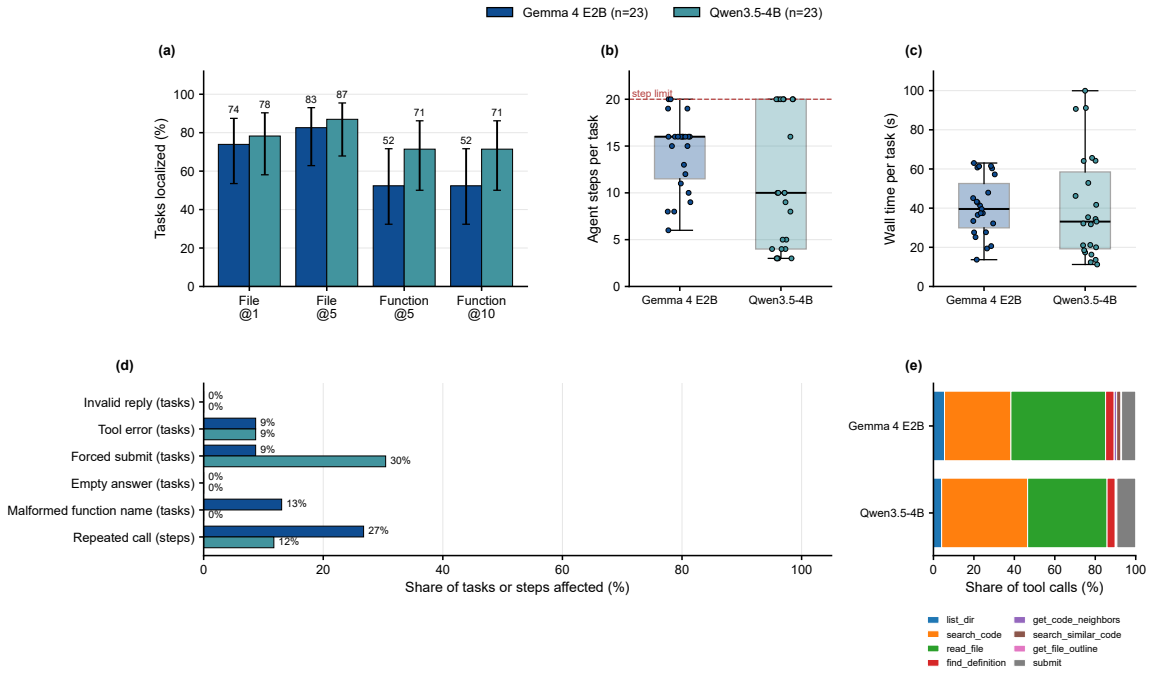


Figure 19: Exp. 10, C2 graph tools (dev). Gemma 4 E2B reaches its best dev accuracy (Function@5 52%); graph tools receive under 10% of calls.

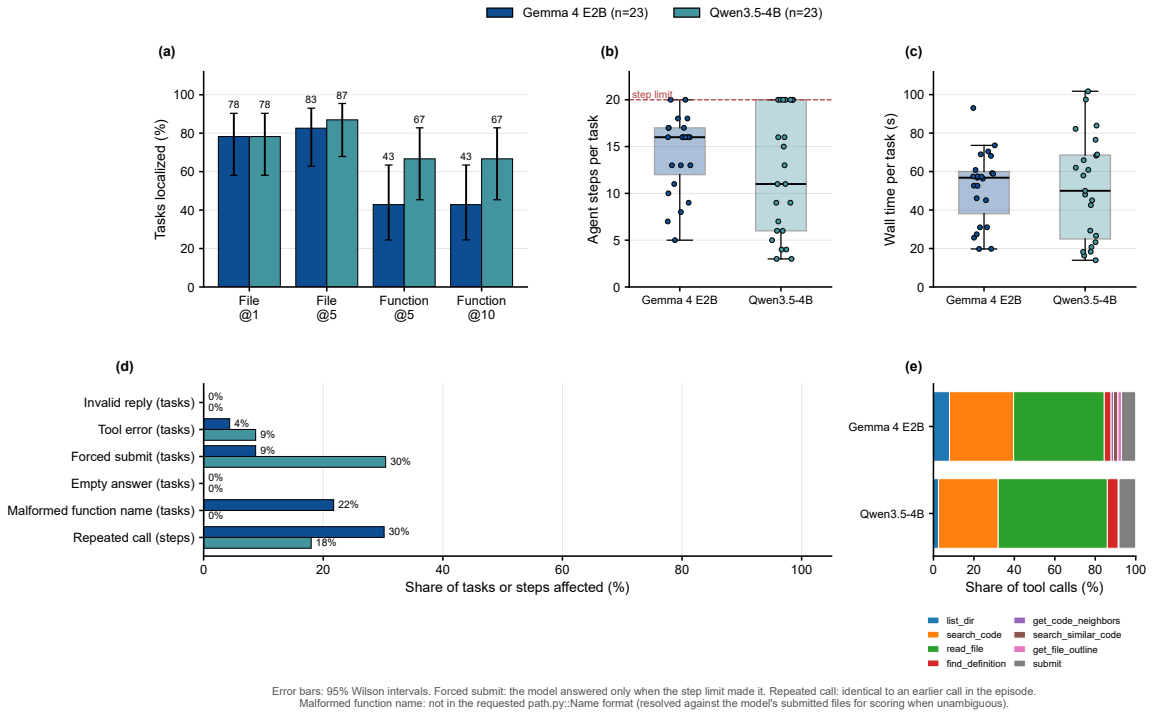


Figure 20: Exp. 11, C3 Evidence Buffer (dev). No accuracy gain, 43–52% more time per task.

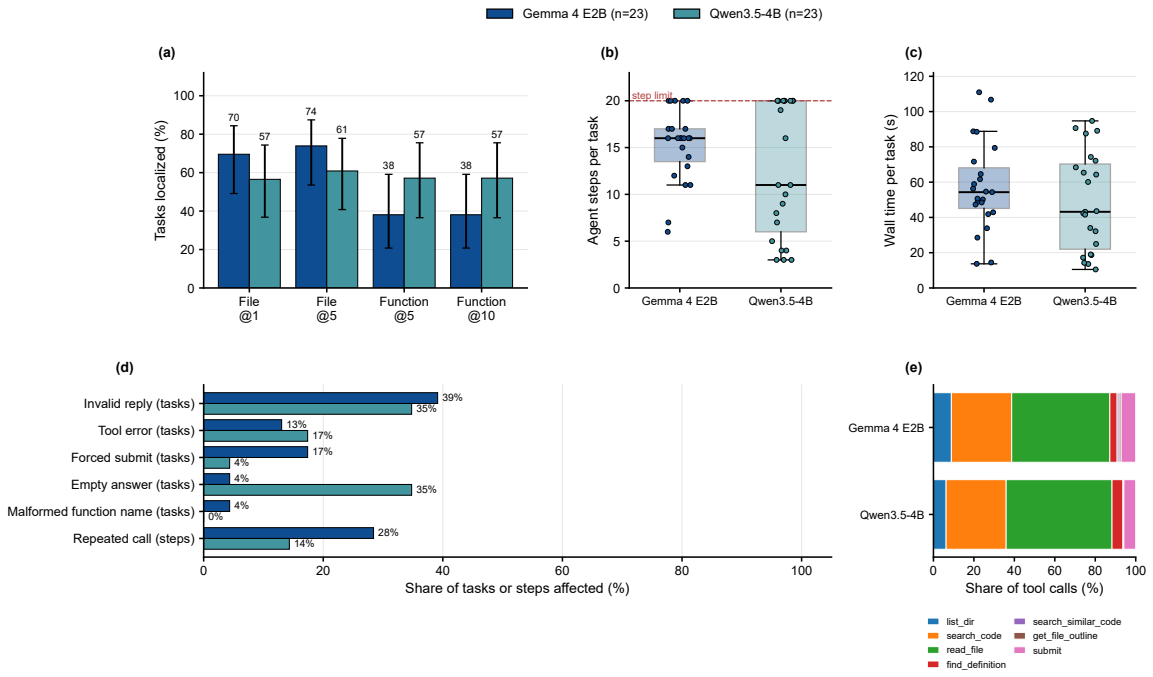


Figure 21: Exp. 12, C3 without grammar (dev). Invalid replies appear in 39% (Gemma 4 E2B) of episodes; Qwen3.5-4B's File@1 falls to 57%.

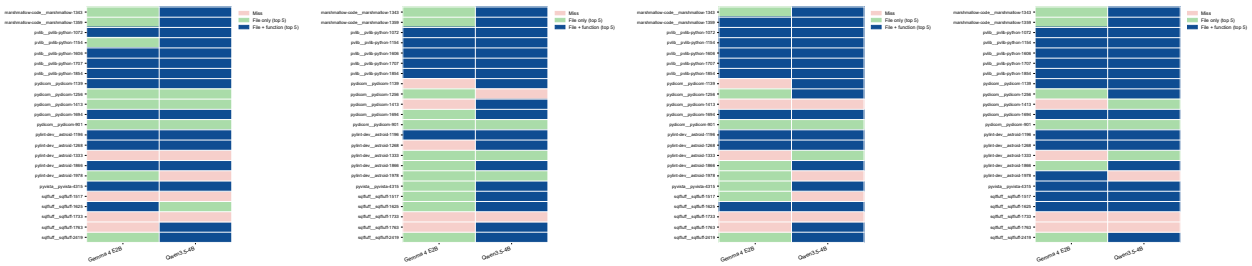


Figure 22: Per-task outcomes on the dev split for experiments 03 (C1), 05 (structured answers), 07 (C1+H) and 10 (C2), left to right.

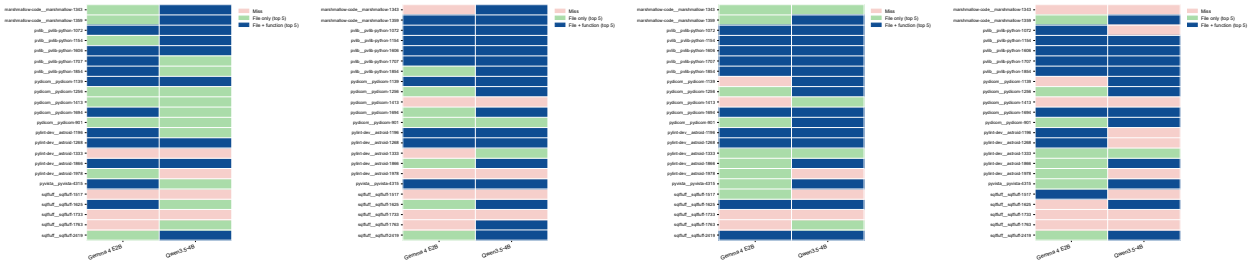


Figure 23: Per-task outcomes on the dev split for experiments 04 (regex names), 06 (guard and budget hints), 11 (C3) and 12 (C3 without grammar), left to right.

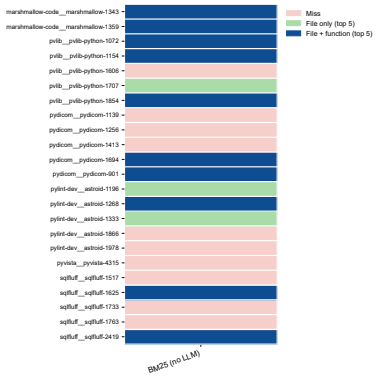


Figure 24: Per-task outcomes of BM25 on the dev split (exp. 09).

B.2 Test split

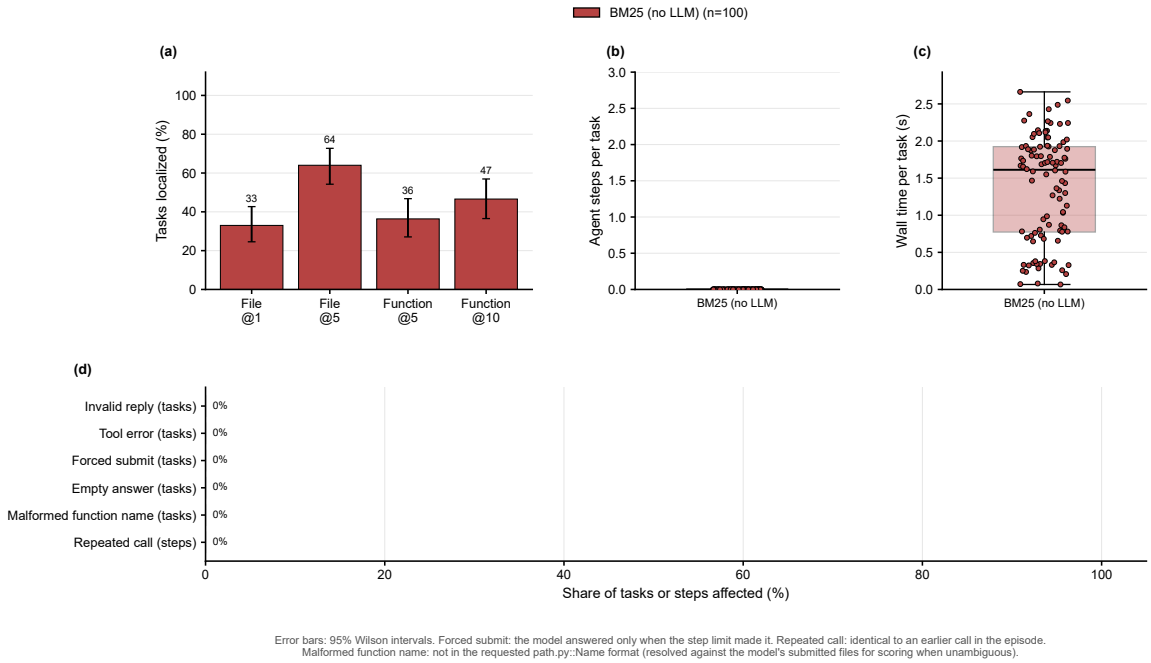


Figure 25: Exp. 13, C0 BM25 (test). 33% File@1, 64% File@5, 36% Function@5, 47% Function@10.

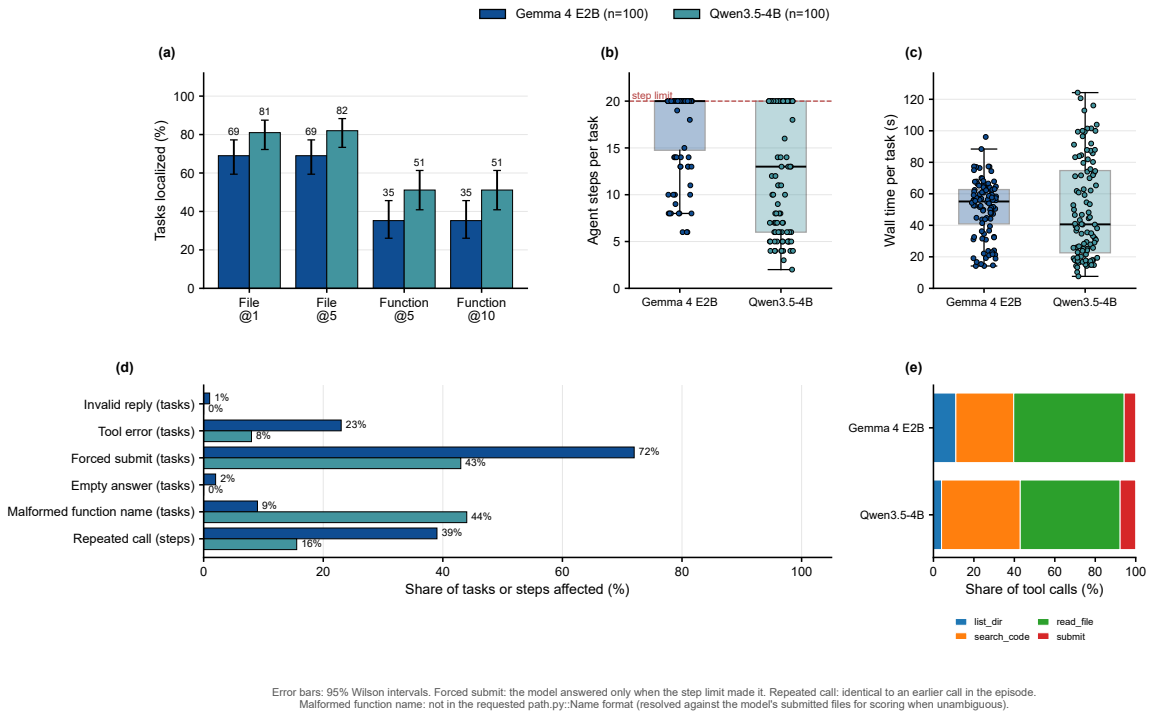
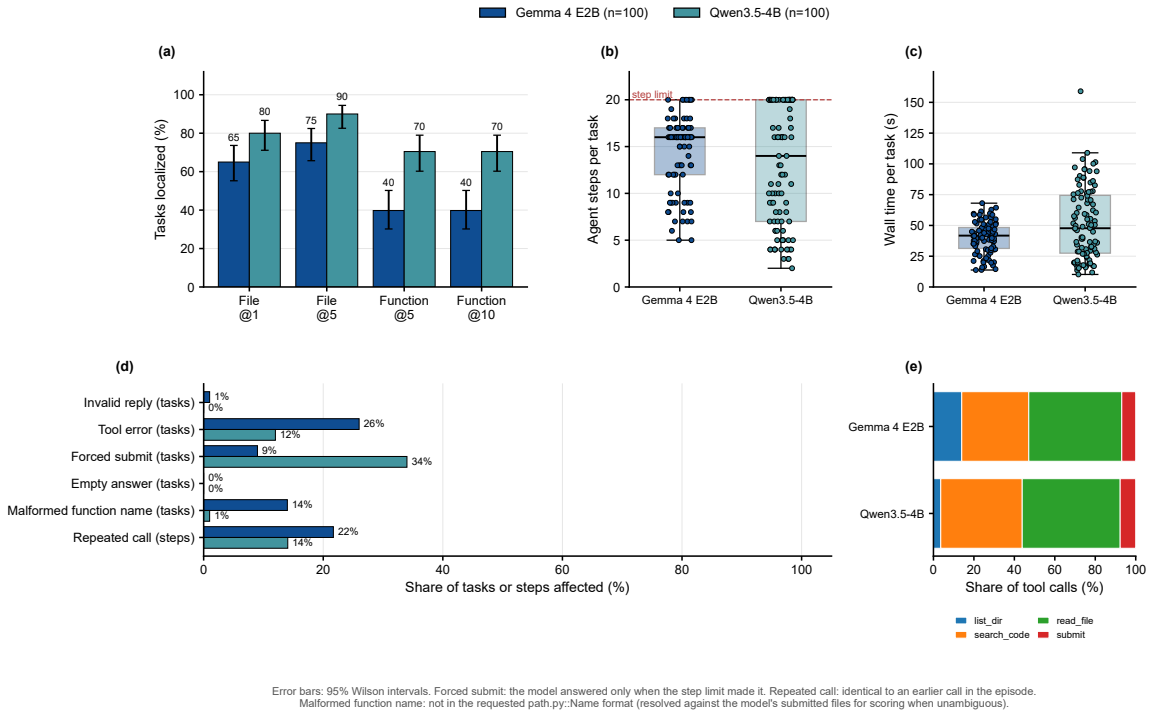
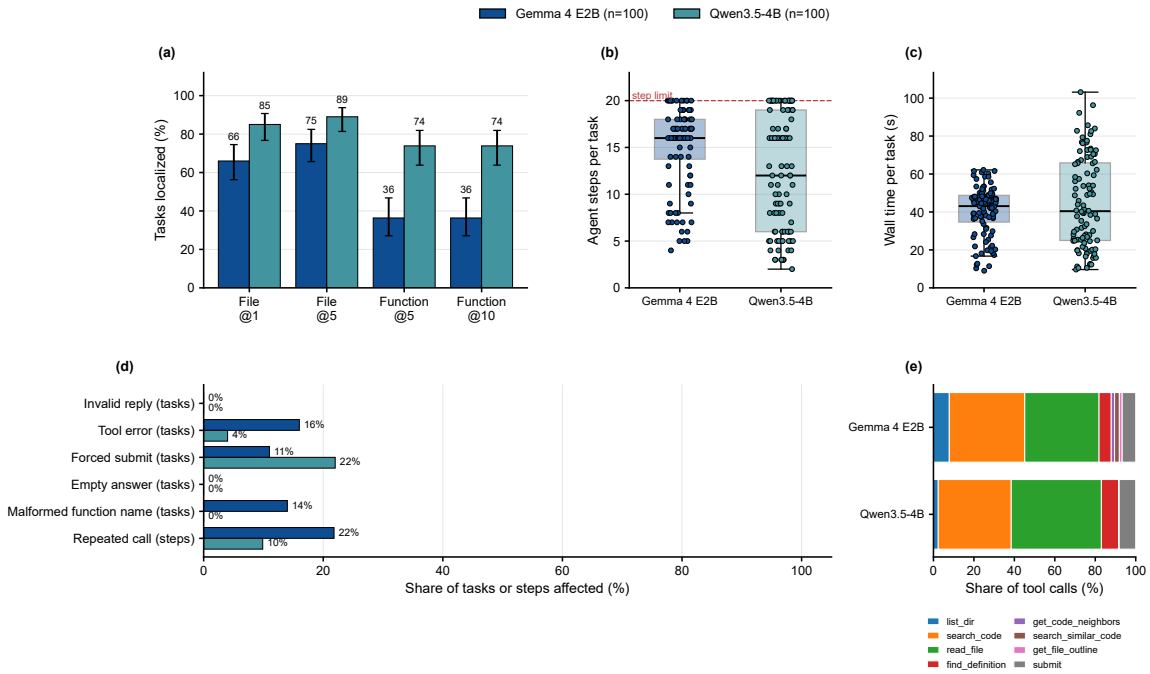


Figure 26: Exp. 15, C1 (test). Gemma 4 E2B hits the step limit in 72% of episodes and repeats calls in 39% of steps; 44% of Qwen3.5-4B's tasks contain malformed function names.



Error bars: 95% Wilson intervals. Forced submit: the model answered only when the step limit made it. Repeated call: identical to an earlier call in the episode.
Malformed function name: not in the requested path.py::Name format (resolved against the model's submitted files for scoring when unambiguous).

Figure 27: Exp. 16, C1+H (test). Qwen3.5-4B Function@5 70% (from 51%); Gemma 4 E2B's forced submissions fall to 9%.



Error bars: 95% Wilson intervals. Forced submit: the model answered only when the step limit made it. Repeated call: identical to an earlier call in the episode.
Malformed function name: not in the requested path.py::Name format (resolved against the model's submitted files for scoring when unambiguous).

Figure 28: Exp. 14, C2 (test), the full system. Qwen3.5-4B: 85% File@1, 74% Function@5; Gemma 4 E2B: 66% and 36%.

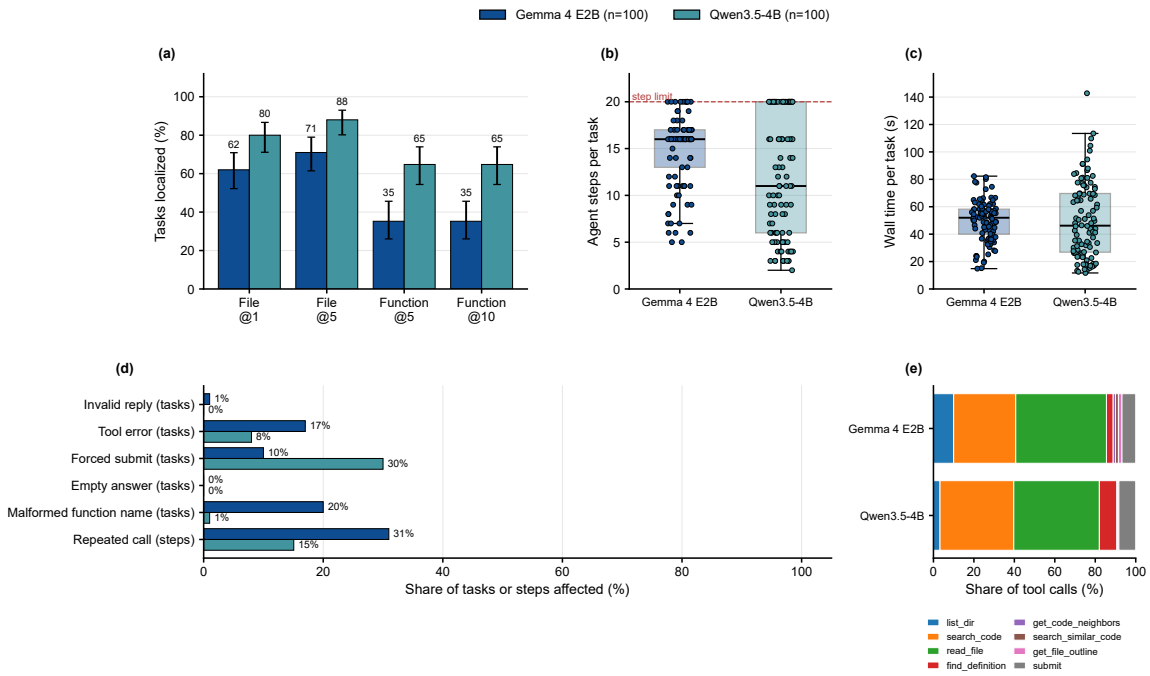


Figure 29: Exp. 17, C3 (test). Every accuracy metric slightly lower than C2, with more generated tokens.

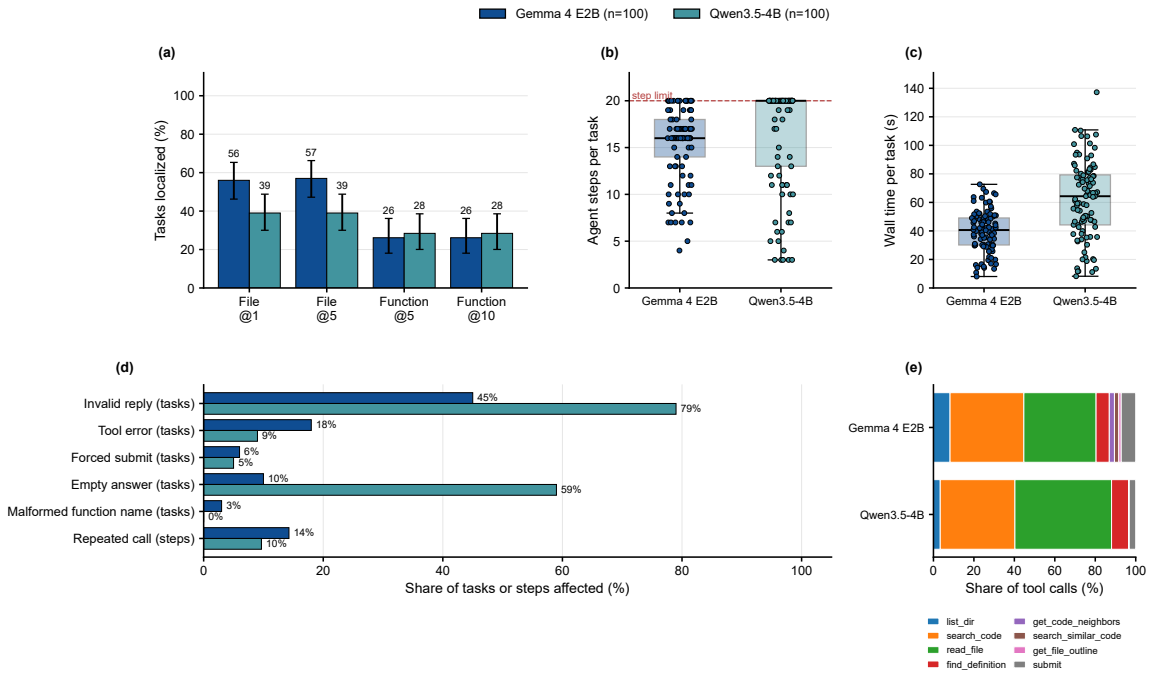


Figure 30: Exp. 18, C2 without grammar (test). Invalid replies in 45% (Gemma 4 E2B) and 79% (Qwen3.5-4B) of episodes; 59% of Qwen3.5-4B's answers are empty.

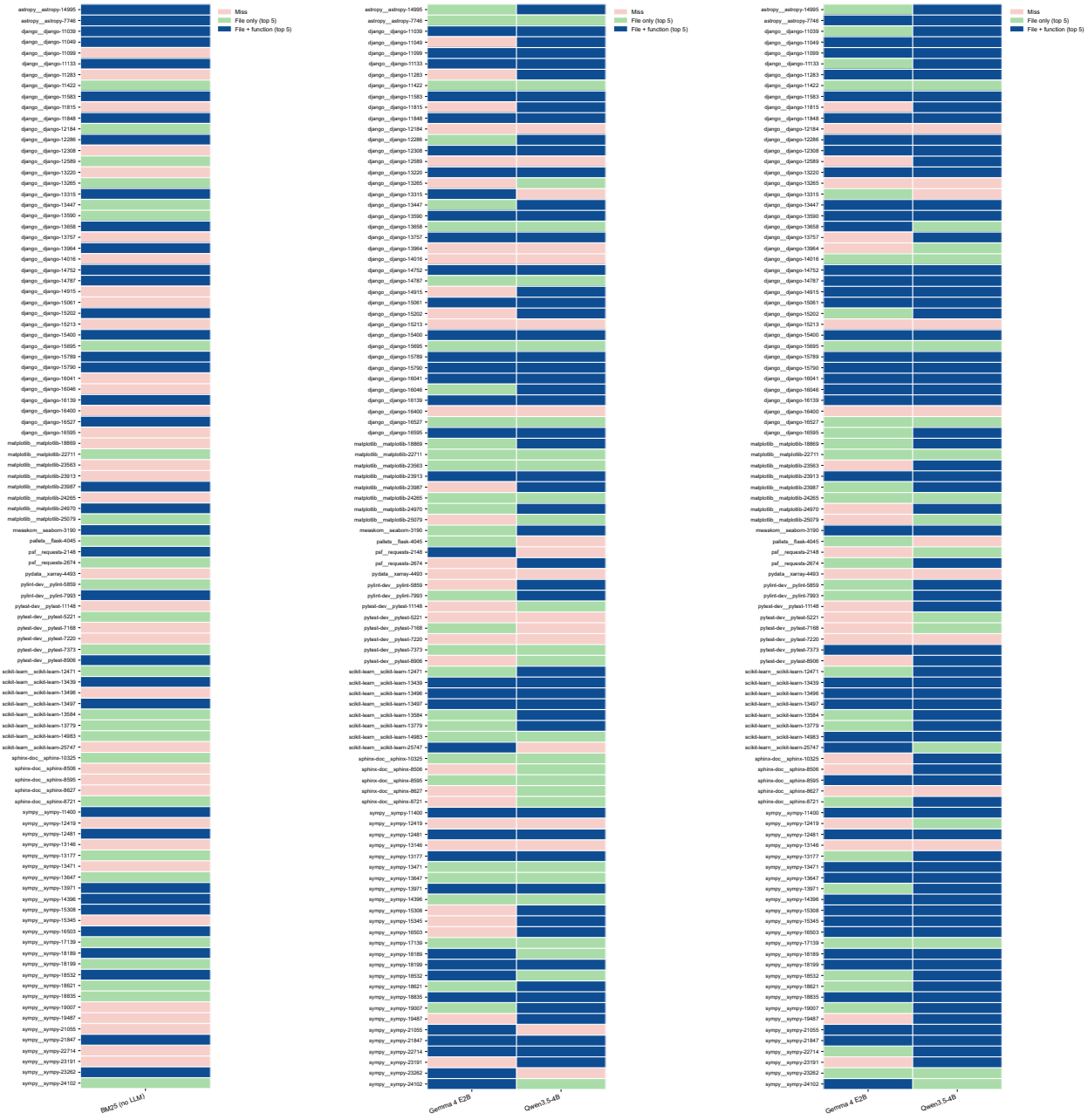


Figure 31: Per-task outcomes on the test subset: C0 BM25 (exp. 13), C1 (exp. 15) and C1+H (exp. 16), left to right.

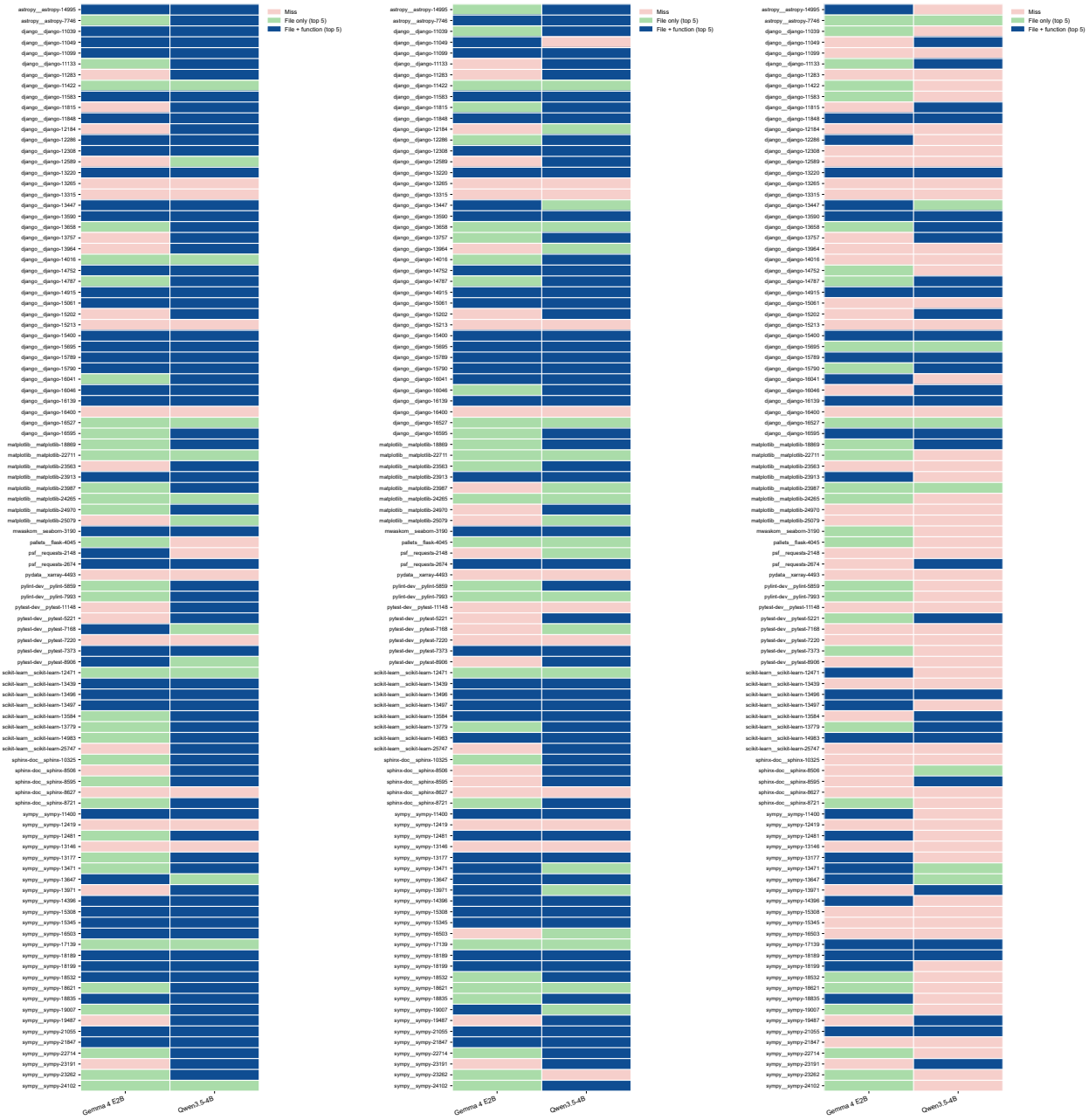


Figure 32: Per-task outcomes on the test subset: C2 (exp. 14), C3 (exp. 17) and C2 without grammar (exp. 18), left to right.